SIH26181 · QUALCOMM HARDWARE CHALLENGE

Beginner's Guide to Understanding Waveforms

Simple analogies connecting digital waveforms to the health companion project

Zynq-7000 | ShrikeFi / ForgeFPGA | 50 MHz Clock Domain | PPG → Heart Rate

For anyone opening GTKWave for the very first time

Contents

How to use this guide: Read it top to bottom once, then open the actual waveform in GTKWave and follow the "story" in Section 4 signal-by-signal. You don't need to know Verilog — every signal here is explained through a hospital/heart-monitor analogy first, and the technical meaning second.

1 What Is a Waveform?

Think of a waveform like a **heart monitor in a hospital** — it shows voltage over time. Each line (signal) is like a separate monitor showing a different aspect of the system.

> **In our project:** The waveform shows what happens inside the FPGA (the hardware accelerator) cycle by cycle, at **50 MHz** (once every **20 nanoseconds**). Everything you'll ever debug in this project happens somewhere on one of these timelines.

Why engineers even bother with waveforms

Software developers use `print()` statements to see what a program is doing. Hardware designers can't do that — a chip doesn't have a console. Instead, a simulator records the value of every wire at every clock tick and draws it as a timeline. That timeline *is* the waveform. Reading one is basically reading a very precise diary of the circuit's life.

2 The Two Platforms — Same Heart, Different Bodies

Both platforms run the **exact same peak-detection logic**. Only the "packaging" around it changes — like the same heart specialist working in a fully-equipped hospital one day, and doing house calls with a small bag the next.

🏥 Zynq-7000 — The "Hospital" Setup

- **FPGA** = Specialized heart-monitoring equipment
- **ARM Processor** = Doctor reading the monitors
- **AXI4-Lite Bus** = Medical chart where the doctor writes orders & reads results

🩺 ShrikeFi — The "Wearable" Setup

- **ForgeFPGA** = Tiny heart-monitor chip
- **ESP32-S3** = Smartwatch processor with WiFi/Bluetooth
- **4-Bit Link** = Ultra-thin wire (only 4 lines!) passing notes between them

> **Why it matters:** If you understand one platform's waveform, you basically understand both — the filter and FSM signals are identical. Only the communication layer at the very end differs (AXI registers vs. a 4-wire nibble link).

3 Signal Groups Explained Simply

3.1 Clock & Reset — The Metronome & Power Switch

Signal	Analogy	What to See
`clk` / `s_axi_aclk`	Metronome ticking at 50 MHz (20 ns per tick)	Perfect square wave — everything else dances to this beat
`rst_n` / `s_axi_aresetn`	Power-on reset button	Starts low (reset), goes high (system alive)

Beginner tip: If the clock isn't toggling, nothing works. If reset stays low, the system never starts. These two signals are the first thing to check whenever a waveform "looks empty."

3.2 Moving Average Filter — The "Smoothing" Machine

Real problem: Raw PPG signal from the finger sensor is noisy (motion, ambient light).
Hardware solution: Average the last 8 samples to smooth it out.

In the waveform:

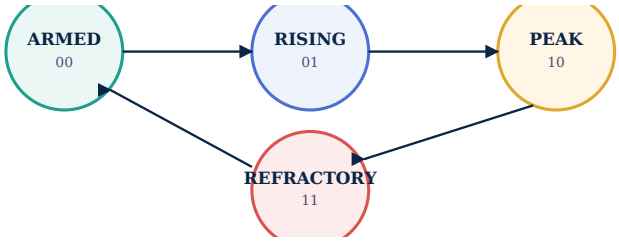
- `reg_red_raw` / `reg_ir_raw` = Raw noisy samples coming in
- `red_filtered` / `ir_filtered` = Clean smoothed output

The magic: Uses `>> 3` (divide by 8 via a wire shift) — **zero multipliers, zero memory**. This runs in **1 clock cycle (20 ns)**. Software doing the same averaging in a microcontroller loop would take microseconds — roughly a thousand times slower.

3.3 Peak Detector FSM — The "Heartbeat Spotter"

Analogy: A nurse watching the smoothed wave, tapping her foot on every beat.

State	Nurse Analogy	Waveform Signal
ARMED (00)	"Waiting for the beat to rise"	`current_state = 0`
RISING (01)	"Going up... going up..."	`current_state = 1`
PEAK (10)	"TOP! Beat detected!"	`current_state = 2` + `beat_pulse` fires
REFRACTORY (11)	"Ignore for 250 ms — can't have another beat this fast"	`current_state = 3`



Key counters:

- `interval_cnt` = Time since last beat (counts 20 ns ticks)
- `refractory_cnt` = Countdown timer (250 ms = 12.5M cycles)

In the waveform: Watch `current_state` cycle 0→1→2→3→0→1→2→3... — once per heartbeat!

3.4 Output: The "Doctor's Notes"

Signal	Meaning	Units
`hw_beat_pulse`	One-cycle pulse when a beat is found	Digital 0/1
`irq_beat`	Interrupt to the processor: "New beat!"	Digital 0/1
`hw_ibi_cycles`	**Inter-Beat Interval** in clock cycles	Cycles × 20 ns
`reg_ibi_latched` (ShrikeFi)	IBI value latched for 4-bit link transfer	Cycles

Calculate heart rate:

```
Heart Rate (BPM) = 60 / (ibi_cycles × 20ns)
                 = 60 / (ibi_cycles × 20 × 10^-9)
                 = 3,000,000,000 / ibi_cycles
```

Worked example: if `ibi_cycles` = 50,000,000, then BPM = 3,000,000,000 ÷ 50,000,000 = **60 BPM**. A slower heart rate means a *larger* ibi_cycles number — more clock ticks pass between beats.

3.5 Communication: How the FPGA Talks to the Processor

Zynq: AXI4-Lite (The Medical Chart)

```
Doctor (ARM)                    FPGA (Monitor)
   |                               |
   |-- Write 0x0C (threshold) --->|  "Set detection sensitivity"
   |                               |
   |-- Read 0x04 (red_filtered) <-|  "Give me the clean red signal"
   |-- Read 0x08 (ibi_cycles)   <-|  "How long since the last beat?"
   |                               |
```

Waveform: Look for `awaddr`/`wdata` (writes) and `araddr`/`rdata` (reads) — with handshake signals like `valid`/`ready` confirming each transfer.

ShrikeFi: 4-Bit Link (The Tiny Wire)

Only **4 wires** + strobe + direction!

```
FPGA                    ESP32-S3
  |                       |
  |-- link_dout[3:0] --->|  Nibble 0: red_filtered[7:4]
  |   (strobe pulses)     |  Nibble 1: red_filtered[3:0]
  |                       |  Nibble 2: ir_filtered[7:4]
  |                       |  Nibble 3: ir_filtered[3:0]
  |                       |  Nibble 4-7: ibi_cycles[31:0] (32 bits = 8 nibbles)
  |                       |
  |<-- link_din[3:0] -----|  Commands from ESP32 (threshold, config)
```

Waveform: Watch `link_dout[3:0]` shift nibbles, `link_strobe` pulse once per nibble, and `link_dir` flip to show which direction data is currently flowing.

Why a 4-bit link at all? AXI4-Lite needs dozens of wires and a lot of power to stay active — great for a hospital device plugged into the wall. A wearable can't afford that. Four wires means fewer pins, less routing, and far lower standby power — exactly what a coin-cell-powered wristband needs.

4 How to Read the Waveform Like a Story

Once you can name the signals, the trick is reading them *in order*, left to right, like a comic strip. Each step below corresponds to a moment you can actually point at on the timeline.

Zynq Story (open `presentation.gtkw`)

1. **Reset releases** → system wakes up
2. **ARM writes threshold** (0x0C) via the AXI write channel
3. **Raw PPG samples arrive** → `reg_red_raw` changes
4. **Filter smooths** → `red_filtered` follows with a 1-cycle lag
5. **Peak detector FSM cycles** → `current_state` 0→1→2→3→0...
6. **Beat found!** → `hw_beat_pulse` + `irq_beat` fire
7. **IBI captured** → `hw_ibi_cycles` updates
8. **ARM reads IBI** (0x08) via the AXI read channel
9. **Scoreboard** → `tests_passed` increments

ShrikeFi Story (open `presentation.gtkw`)

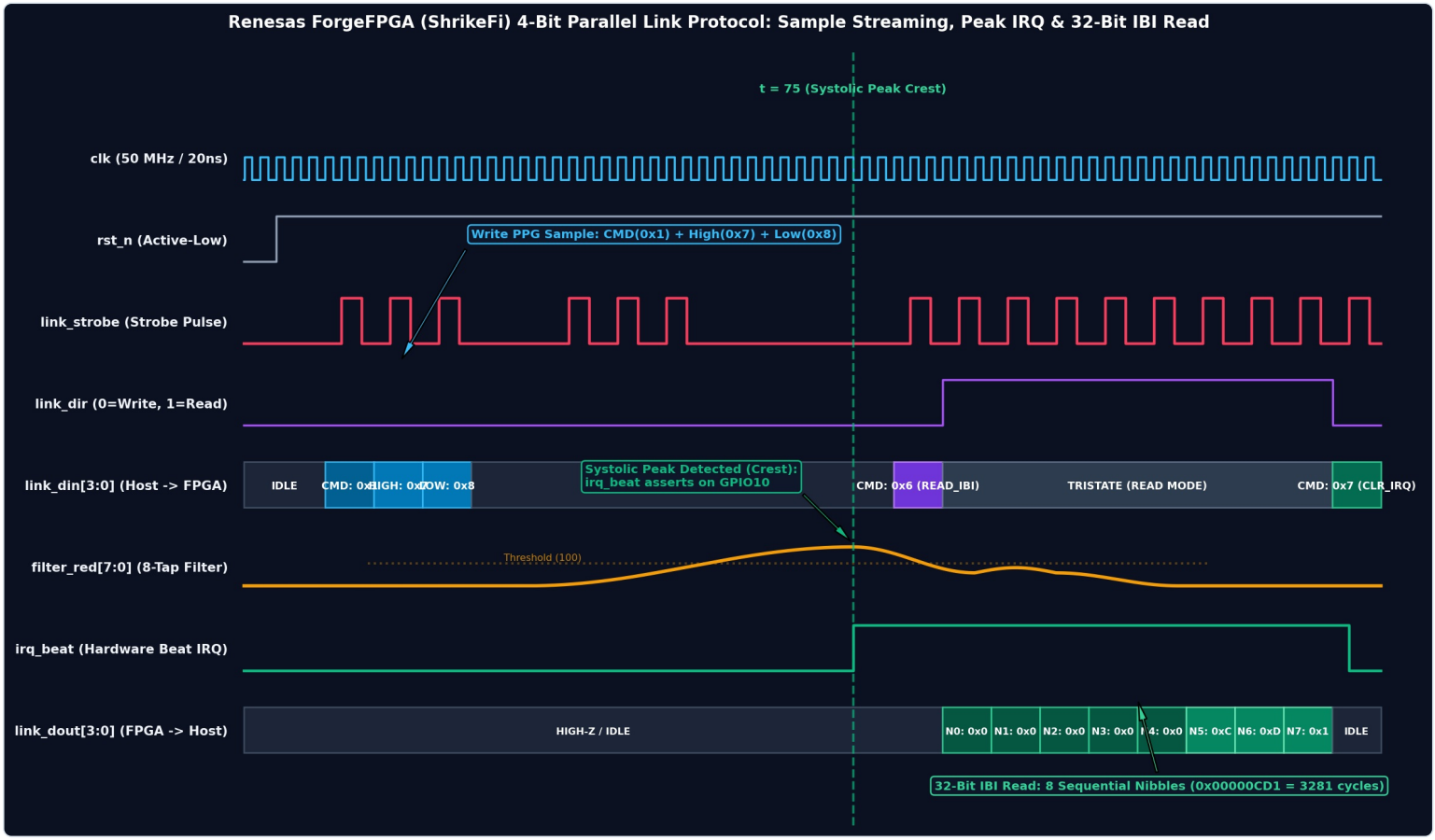
1. **Reset releases** → both sides wake up
2. **Link synchronizes** → `strobe_sync` aligns
3. **Raw PPG → Filter → Peak detect** (identical to the Zynq flow!)
4. **Beat found** → `peak_beat_detected` fires
5. **IBI latched** → `reg_ibi_latched` holds the value
6. **4-bit link transmits** → `link_dout` shifts out 12 nibbles (red + ir + ibi)
7. **ESP32 receives** → `link_din` collects the nibbles
8. **Interrupt sent** → `irq_beat` to the ESP32
9. **Scoreboard** → `tests_passed` increments

> **Reading trick:** pin your GTKWave cursor to step 6 (beat found) first — everything before it is "setup," and everything after it is "reporting the result." Once you can spot that one moment, the rest of the story locates itself around it.

5 Reading a Real Capture

Everything so far has been a simplified sketch. Below are two **actual annotated captures** from this project's own simulations — one per platform. Same story you just read in Section 4, now with real signal names, real hex values, and a real detected heartbeat.

5.1 ShrikeFi (ForgeFPGA) — 4-Bit Parallel Link Capture

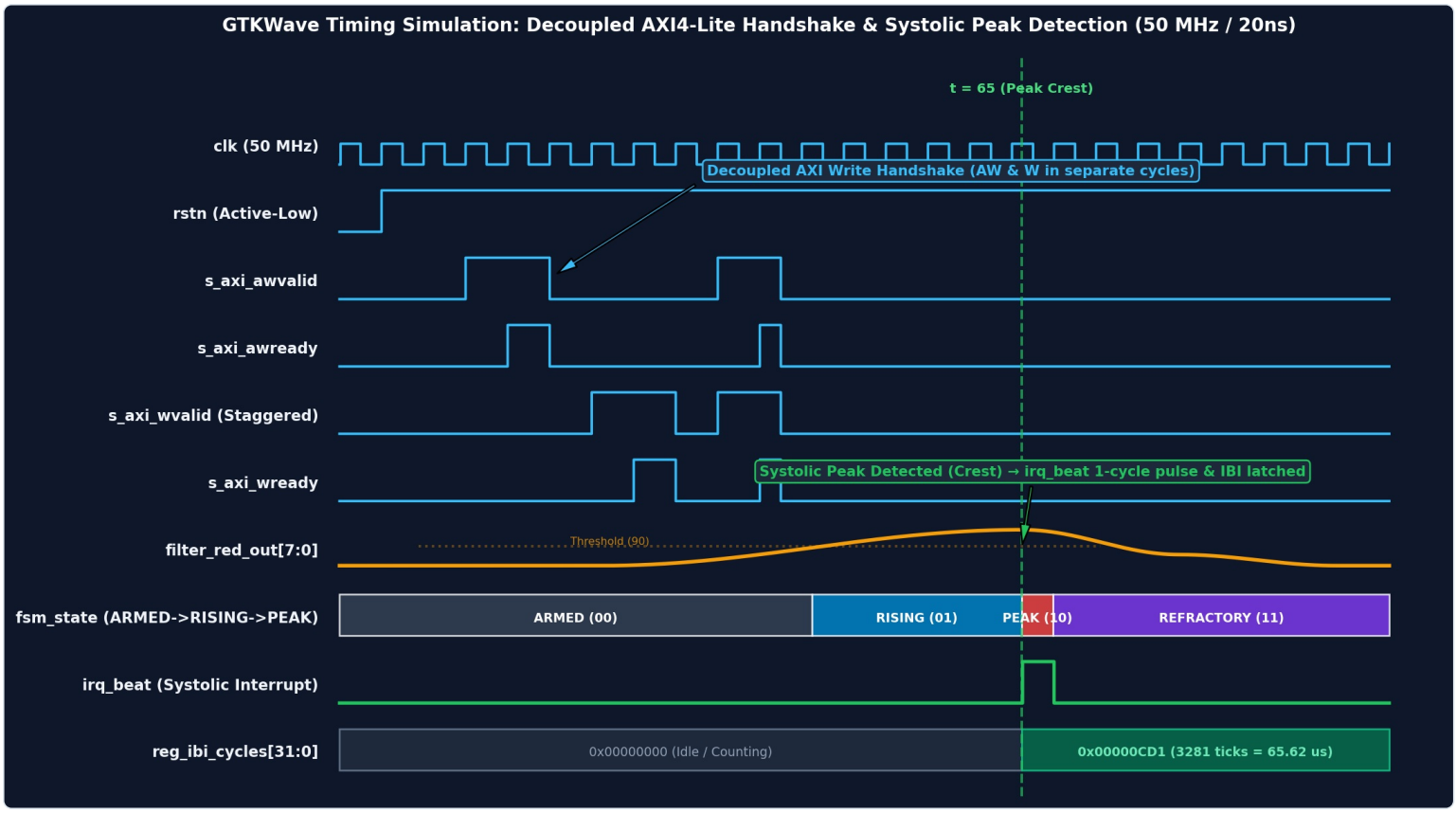


Read this one left to right as three acts: **write the sample in, wait for the peak, read the IBI back out.**

1. `clk` **and** `rst_n` — the 50 MHz metronome is already running before anything interesting happens; `rst_n` goes high once, near the very start, and stays there. That's your "system is alive" confirmation.
2. **Sample write-in (left third)** — watch `link_strobe` pulse three times in a cluster while `link_din[3:0]` shows `CMD: 0x1`, then `HIGH: 0x7`, then `LOW: 0x8`. That's the host writing one PPG sample as three nibbles: a command byte saying "here comes a sample," then the high nibble, then the low nibble. This repeats in bursts — each burst is one new raw sample arriving.
3. **The filter curve** — `filter_red[7:0]` is drawn as a smooth orange curve underneath, climbing toward the dotted `Threshold (100)` line. This is the same moving-average output from Section 3.2, just plotted as a continuous line instead of individual samples.
4. **t = 75, Systolic Peak Crest** — the dashed green line marks the exact cycle the filtered curve tops the threshold. At that instant `irq_beat` jumps high (bottom-middle track) and stays high — this is the hardware telling the host "I found a heartbeat," using the exact ARMED→RISING→PEAK logic from Section 3.3.
5. `link_dir` **flips (0→1)** — right after the peak, direction switches from Write to Read. The link is now letting the FPGA talk instead of the host.
6. **Read-back command** — `link_din[3:0]` shows `CMD: 0x6 (READ_IBI)`, then goes to `TRISTATE (READ MODE)` — the host lets go of the wire so the FPGA can drive it.
7. **32-bit IBI shifted out, one nibble at a time** — `link_dout[3:0]` ticks through `N0` through `N7`, spelling out `0x00000CD1` nibble by nibble. Converting hex to decimal: 0x00000CD1 = **3281 clock cycles** since the last beat.
8. `CMD: 0x7 (CLR_IRQ)` — the very last write on the link. The host acknowledges the interrupt and clears it, resetting the FPGA to listen for the next beat.

> **Do the math yourself:** 3281 cycles × 20 ns/cycle = 65.62 µs between beats?? That seems far too fast for a real heartbeat — and it is. This is a *simulation testbench* feeding synthetic PPG data at simulation speed, not a live 60–100 BPM human pulse. In real operation the same math applies, just with an IBI in the tens of millions of cycles (matching the worked example in Section 3.4).

5.2 Zynq / Artix-7 — Decoupled AXI4-Lite Capture



This capture is from the AXI4-Lite side (Zynq-7000 / Artix-7 class FPGA fabric) — same underlying peak-detector RTL as ShrikeFi, but talking over a real AXI bus instead of a 4-wire link.

1. `clk` **and** `rstn` — identical role to the ShrikeFi capture: clock free-runs, reset releases once near the start.
2. **Decoupled AXI write handshake** — notice `s_axi_awvalid` pulses high, and `s_axi_awready` pulses high a cycle *later*, not at the same instant. Same story for `s_axi_wvalid` and `s_axi_wready` just after. This is exactly the valid/ready handshake from Section 3.5, except AXI4-Lite is allowed to send the address (AW) and the data (W) in separate, staggered cycles — that's what "decoupled" means here. You'll see this pattern repeat twice: two separate register writes (e.g. setting the threshold, then another config value).
3. `filter_red_out[7:0]` — the same smoothed PPG curve as before, this time climbing past a `Threshold (90)` line.
4. `fsm_state` **track** — this is the one signal in this whole guide you should stare at the longest. It's drawn as a literal state strip: `ARMED (00)` for most of the timeline, then a block of `RISING (01)`, a thin sliver of `PEAK (10)` right at the crest, then `REFRACTORY (11)` afterward. This is Section 3.3's state diagram, but drawn as it actually plays out in time — notice how much longer ARMED and REFRACTORY last compared to the brief PEAK state.
5. **t = 65, Peak Crest** — the dashed green marker lines up exactly with the ARMED/RISING/PEAK boundary. At this exact cycle, `irq_beat` fires a single one-cycle pulse (visible as a narrow spike, not a level that stays high) — that one-cycle-only shape is what `hw_beat_pulse` means in Section 3.4: it announces the event once and drops back down immediately.
6. `reg_ibi_cycles[31:0]` — sits at `0x00000000 (Idle / Counting)` the entire time leading up to the peak, then latches to `0x00000CD1 (3281 ticks = 65.62 us)` the instant the peak fires. Same hex value as the ShrikeFi capture — confirms both platforms are running the identical peak-detector logic on identical test data, exactly as Section 2 promised.

Compare the two captures side by side: the ShrikeFi version spreads the same information across nibble-sized writes on 4 wires over many cycles; the AXI version moves it in wider parallel words over a richer bus in fewer cycles. Same heartbeat, same 3281-cycle IBI, same hex value at the end — just a different "shipping method" for the data, matching the hospital-vs-wearable analogy from Section 2.

6 Beginner Exercises

Do these in order — each one builds on the reading skill from the last.

Exercise 1 — Count Heartbeats

1. Open the Zynq waveform
2. Find `hw_beat_pulse` (green group)
3. Count rising edges in a 2-second simulation window
4. Calculate: `beats / 2s × 60 = BPM`

Exercise 2 — Measure Filter Latency

1. Find `reg_red_raw` and `red_filtered` (purple group)
2. Place a cursor on the raw sample change
3. Place a cursor on the filtered output change
4. Difference = **1 clock cycle (20 ns)**

Exercise 3 — Trace FSM States

1. Find `current_state` (red group)
2. Watch the sequence: 0 → 1 → 2 → 3 → 0 → 1 → 2 → 3...
3. Each full cycle = **one heartbeat detected**

Exercise 4 — Decode the 4-Bit Link (ShrikeFi)

1. Find `link_dout[3:0]` and `link_strobe` (magenta group)
2. Count strobe pulses per beat transfer
3. Should be ~12 nibbles (3 bytes red + 3 bytes ir + 4 bytes ibi + overhead)

Bonus Exercise 5 — Break It On Purpose

1. In the testbench, hold `rst_n` low for an extra 10 cycles
2. Re-run the simulation and reload the waveform
3. Confirm every signal stays flat/zero until reset actually releases
4. This is the fastest way to learn what "broken" looks like before you see it by accident

7 Connecting to the Big Picture

Waveform Signal	Project Feature	Why It Matters
`red_filtered`	Clean PPG for SpO2	Noisy signal → wrong oxygen reading
`hw_ibi_cycles`	HRV (Heart Rate Variability)	20 ns resolution → detects stress 15–30 min early
`irq_beat`	Real-time alert	Processor wakes only on a beat, saving power
`link_dout` (ShrikeFi)	Ultra-low-power comms	4 wires vs. 100+ for AXI → wearable-size form factor

8 GTKWave Quick-Start (bonus)

If this is your first time opening GTKWave, here's the two-minute version:

1. Launch with `gtkwave presentation.gtkw` — the pre-built config already loads the right signals and colors
2. The left panel is the **signal tree**; the right panel is the **timeline**
3. Click a signal name, it highlights on the timeline — use this to match names in this guide to lines on screen
4. Use **Zoom → Fit** (or press `Ctrl+Shift+F`-style shortcuts depending on build) to see the whole simulation at once before zooming into one heartbeat
5. Left-click on the timeline to drop a cursor; a second click elsewhere lets you read the time delta between two points — this is exactly how you'll do Exercise 2
6. Multi-bit buses like `link_dout[3:0]` show as hex by default — right-click → *Data Format* to switch to binary if you want to see individual bits

9 Mini Glossary (bonus)

`Clock domain`

Everything that changes in step with one particular clock signal — here, the 50 MHz FPGA clock.

`Register (reg)`

A small piece of memory that holds one value between clock edges — think of it as a sticky note that gets rewritten once per tick.

`FSM (Finite State Machine)`

A circuit that is always in exactly one of a fixed set of "states" (like ARMED, RISING, PEAK, REFRACTORY) and moves between them based on rules.

`Handshake (valid/ready)`

A pair of signals used so two circuits agree data is actually being transferred, not just sitting on the wire.

`Latch / latched value`

A value that gets "frozen" and held steady so a slower reader (like the ESP32) has time to grab it before it changes again.

`Nibble`

Half a byte — 4 bits. The ShrikeFi link moves one nibble per strobe pulse because it only has 4 data wires.

`Testbench`

Extra code that isn't part of the real chip — its only job is to feed fake sensor data in and check the outputs during simulation.

`VCD file`

The raw recording of every signal's value over time — the file GTKWave actually reads to draw the waveform.

10 Files Reference & Next Steps

File	Purpose
`ppg_system.vcd`	Zynq simulation dump
`hardware/shrikefi/shrikefi_sim.vcd`	ShrikeFi simulation dump
`hardware/zynq/presentation.gtkw`	Zynq presentation config
`hardware/shrikefi/presentation.gtkw`	ShrikeFi presentation config
`docs/WAVEFORM_PRESENTATION_GUIDE.md`	Signal tables for presentations
`hardware/common/moving_average_8tap.v`	Filter RTL (readable!)
`hardware/common/ppg_peak_detector.v`	Peak detector RTL (readable!)

1. Open both waveforms side by side in GTKWave
2. Compare filter outputs — same RTL, different platforms
3. Compare peak detectors — same FSM, different interfaces
4. Read the RTL — it's shorter than you think!
5. Modify the threshold in the testbench and watch detection change

Remember

Every signal in the waveform maps to a line of Verilog or C code. The waveform is just the code "running in slow motion" so humans can understand it.